Learning to Apply, Not to Memorize: Policy-in-Context Fine-Tuning Generalizes to Unseen Organizational Policies

Anonymous ACL submission

Abstract

The people who own a task in an organization are rarely programmers or model experts, and the documents that govern their work—policies, runbooks, checklists—change on the organization's schedule. We build on a simple workflow: a worker first solves the job in conversation with an agent; the recorded sessions are expressed in the declarative work language OpenWorkLang and compiled by OpenWorkCompiler, so that policies become an ontology with rules, calculations become programs, and judgment goes to a small local model. This paper tests the doubtful link in that chain: can a small model learn the **skill** of judgment—applying whatever policy it is given in context rather than memorizing any particular policy? On 34 organizational decision cases (3,400 decisions labeled by a deterministic rules engine), a QLoRA-tuned 7B model reaches **72.9%** exact match (verdict, routing, parameters, and a *correct* cited rule) on policies never seen in training, evaluated leave-case-out so every policy is unseen exactly once (cluster-bootstrap 95% CI $[64.7, 80.9]$), versus 15.0% for the untrained model under identical serving. Controls pin down what was learned: **88.0%** on counterfactual policies where prior knowledge scores zero by construction, 50.0% for a 3-shot in-context-learning baseline, $\pm$4.8pp variance across training seeds. A negative control bounds the recipe: the same recipe on byte-exact arithmetic scores 0/6—with chain-of-thought targets, and even for a tool-less frontier model—so calculation is compiled to code once rather than delegated to an LLM on every run. Because compilation starts from a work session the organization already paid for, the frontier-token break-even is the **second run**.

1 Introduction

In most organizations, correct behavior is defined by *documents*—decision policies, support runbooks, escalation matrices, grading rubrics, editorial style guides, compliance checklists—and those documents change on the organization's schedule, not the model's. Any AI system built on them needs a particular decomposition: the **content** (what the document says today) must live *in context*, where it can be swapped the day it changes, while the **skill** (how to read such a document and apply it faithfully to a case) can live *in weights*. This paper measures whether that skill is actually learnable by supervised fine-tuning of a small model—and whether it transfers to documents the model has never seen.

In practice the decomposition plays out like this. The people who own these tasks are not programmers or model experts. At first they simply do their job—quotes, refund approvals, incident triage—in conversation with an agent. Those working sessions already contain the job's policies, its calculations, and its way of judging. Expressed in a declarative work language (OpenWorkLang) and run through its compiler (OpenWorkCompiler), the policies become an ontology with rules, the calculations become programs, and the judgment goes to a trained small model—yielding a task-specific execution-and-decision tool that runs on a local GPU, without a frontier LLM, at a fraction of the tokens. This paper puts the most doubtful link of that chain to the test—is judgment actually learnable by a small model?—and backs the other links with controls and measurements (calculation belongs to programs, §5.3; the token economics, §5.4).

We study the cleanest instance of this class that still admits a deterministic grader: organizational decision policies. When an employee asks "can I give this customer a 12% discount?", the organization does not want an oracle's opinion. It wants *its own policy applied*: the ordered rules checked top-down, the first matching rule cited, the case routed to the approver the policy names, and the band

the policy deliberately left open ("5–10%: AI may recommend, team lead approves")—and only that band—filled by a recommendation. Three properties shape the ML problem, each instantiating the general decomposition:

1. **The documents change faster than models.** A model that memorized policy P is wrong the day P becomes P'. The content must live *outside* the weights.

2. **Judgments must be auditable.** "Approve" is not an answer; "approve, route to director, citing `strategic_retention_band` whose condition holds on this record" is.

3. **Discretion is declared, not inferred.** The document says where the model may recommend; confidence thresholds do not.

Prior work internalizes policies into weights (Wang et al., 2025), benchmarks agents given domain policy guidelines (Yao et al., 2024; Zhou et al., 2024), or measures robustness to rule updates (Diao et al., 2025). What is missing is the combination deployment actually needs: **fine-tune a small, locally servable model on the *transferable skill* of applying an arbitrary in-context policy, and measure transfer to policies never seen in training.**

Declarative decision policies make the skill (ordered evaluation, first-match, grounded citation) non-trivial while keeping correctness checkable to the byte. Whether the recipe transfers to messier members of the document class is an open question we scope explicitly, with a roadmap, in §6.

Our contributions:

- **Task & corpus** (§3): 34 decision cases across 10 organizational functions, each declared as an ontology plus ordered rules with explicit discretion bands, compiled by a deterministic engine into 3,400 labeled decisions with cited rules and grounded conditions.

- **A strict deterministic grader** (§3.3): exact match on verdict, route, parameters *and* cited rule, plus a check that the cited rule's condition actually holds on the record.

- **The transfer result** (§5.1): leave-case-out over all 34 policies, 72.9% exact $[64.7, 80.9]$ vs. 15.0% raw under identical bf16 greedy serving; ICL ladder $15.0 \rightarrow 50.0 \rightarrow 72.9$.

- **The counterfactual control** (§5.2): 88.0% on flipped policies where priors score zero by construction.

- **Negative control and placement economics** (§5.3–5.4): the same recipe on byte-exact arithmetic fails (0/6), survives a CoT-target ablation, and is matched by a tool-less frontier model failing the same gate—justifying compilation to code over per-run tool calls, with a measured second-run break-even.

2 Related Work

Policy internalization vs. policy-in-context. Multimodal Policy Internalization (Wang et al., 2025) trains policies *into* model parameters so the policy need not be in context at inference—the opposite maintenance bet from ours. Its "Policy Override" setting supplies an updated policy in context at inference as a *generalization probe* of the internalized model; we make in-context application the training objective itself. Policy Reasoning Traces (Imperial and Madabushi, 2025) improves compliance assessment on specific policies such as HIPAA and GDPR with per-policy instance-level train/test splits; cross-policy transfer appears only as an auxiliary analysis, not as the held-out-policy protocol we use. GuideBench (Diao et al., 2025) benchmarks robustness to rule updates.

Policy-following benchmarks. τ-bench (Yao et al., 2024) evaluates state-of-the-art agents given domain policy guidelines; RuleArena's (Zhou et al., 2024) findings distinguish failures of rule identification from failures of the computations the rules require—directly anticipating our §5.3; CRMArena-Pro (Huang et al., 2025) assesses agents across enterprise scenarios. A contemporaneous enterprise benchmark (Yu et al., 2026) independently builds rule-engine-judged decisions; it is evaluation-only and has no discretion bands. DMN (Object Management Group, 2024) has long standardized ordered decision rules with FIRST-hit semantics; our DSL is deliberately in that family—the ML contribution is the transfer measurement, not the rule language.

SFT and arithmetic. Faith and Fate (Dziri et al., 2023) shows Transformers fail at compositional multi-step arithmetic; Chu et al. (2025) shows SFT tends to memorize and struggles out-of-distribution; PAL (Gao et al., 2022) offloads the

solution step to a Python interpreter. Our contribution is the *pairing*: the same recipe, corpus scale and gate that fail on arithmetic succeed on rule application—and a tool-less frontier model fails the same arithmetic gate, sharpening "small models can't compute" into "inference can't compute."

Small-model deployment. Belcak et al. (2025) argue small models are the future of agentic AI and outline an LLM-to-SLM conversion algorithm; we supply a trained-skill recipe and transfer measurement for one high-value slot (policy application) and a deterministic gate that even frontier models fail for another (derivation), yielding a placement rule rather than a preference.

3 Task and Corpus

3.1 Decision cases

A *case* declares one recurring organizational decision: the deciding role, an **ontology** (entities, attributes, relations), a feature space, **ordered rules** (`when` condition → verdict, route, parameters, rationale), optional **discretion bands** (`defer: slm_recommend` with the band's limits as parameters), and a fallback. The corpus holds **34 cases across 10 functions** (sales, finance, support, HR, procurement, legal, IT operations, marketing, logistics, security). A deterministic engine samples records from each case's feature space and applies first-match semantics to produce **100 labeled decisions per case** (3,400 total), each with the applied rule's name, its condition, and a rationale. Generation is seeded and byte-reproducible.

3.2 Policy-in-context format

Per instance the model receives: a system turn naming the organization, role and decision; the case's ontology and its *full ordered rule list* (YAML, verbatim); and the record. It must output one JSON object `{verdict, route, params, cited_rule, rationale}`. Nothing about any specific policy is in the weights; a rule edit changes behavior with no retraining.

3.3 Grading

An answer is **exact** iff verdict, route and parameters equal the engine's decision *and* the cited rule is the first-matching rule *and* that rule's condition actually holds on the record (the last check fails fabricated grounds even when the verdict is guessed right). A **relaxed** variant accepts a cited rule that is not first-match but whose condition genuinely holds with all other fields correct. We also report verdict-only accuracy. Because held-out items cluster by policy, all confidence intervals are **case-cluster bootstrap** (10,000 resamples over cases).

3.4 Honest scope

The corpus is synthetic and self-graded: the engine that labels is the engine that grades, the policies are already machine-readable, and rule-condition field names match record keys—the entity-alignment difficulty of prose policies is absent, as are noisy fields and annotator disagreement. We claim transfer of *policy application on declarative policies*, a deliberately clean substrate; §6 discusses the path to prose policies.

4 Experimental Setup

Model & training. Qwen2.5-7B-Instruct, QLoRA (r=16, α=32, dropout 0.05, all linear projections), completion-only loss, 2 epochs, LR 10^{-4} cosine, bf16 compute on one RTX 4090 (≈12 min/run). Training data: 28 cases × 40 instances (1,120 rows); 6 cases held out *entirely*. The environment is pinned and every run writes a manifest (seed, epochs, data and adapter SHA-256, package versions).

Serving & fairness. All models—tuned *and* raw baselines—are evaluated through the same bf16 server with greedy decoding. (Re-running raw baselines under identical serving moved raw 7B from 16.7% to 15.0%; the quantization confound does not explain the gap.)

Splits. (a) *Pinned*: 6 held-out cases × 10 instances, plus 56 seen-policy/unseen-instance items. (b) *Leave-case-out (LOCO)*: 6 folds retrained with the same recipe so every one of the 34 policies is unseen exactly once (340 items, 34 clusters).

5 Results

5.1 Transfer to unseen policies

LOCO fold accuracies range 65.0–80.0%: the pinned-split result is not an artifact of one partition (Table 1, Figure 1a). Across 3 training seeds, unseen exact is $62.8\% \pm 4.8$pp—run variance is real and reported; no seed approaches the raw baseline. In-context examples help substantially ($15 \rightarrow 50\%$) but fine-tuning adds ∼20pp over ICL

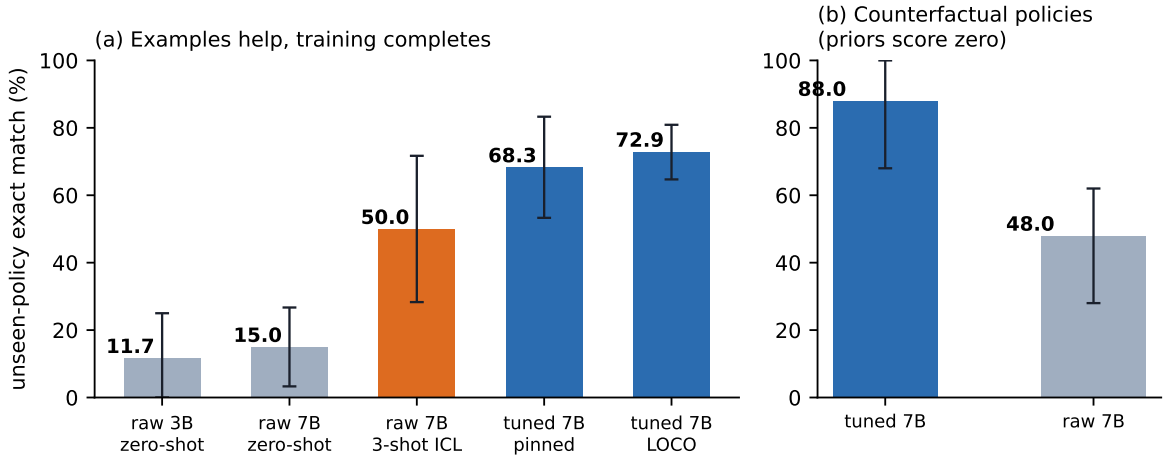

Figure 1: (a) Unseen-policy exact match with case-cluster bootstrap 95% CIs. (b) The counterfactual control: on records where the flipped policy decides differently, priors score zero by construction.

model (identical serving)	exact	95% CI
raw 3B, zero-shot	11.7%	[0.0, 25.0]
raw 7B, zero-shot	15.0%	[3.3, 26.7]
raw 7B, 3-shot ICL	50.0%	[28.3, 71.7]
tuned 7B (pinned split)	68.3%	[53.3, 83.3]
tuned 7B (LOCO)	**72.9%**	**[64.7, 80.9]**

Table 1: Unseen-policy exact match. LOCO holds every one of the 34 policies out exactly once.

arm	gate	field acc.
SFT 7B (plain targets)	0/6	—
SFT 7B (CoT targets)	0/6	86.4%
frontier, same prompt, no tools	0/6	93.6%

Table 2: The arithmetic negative control. Both tuned and frontier arms fail almost solely on two long-division fields.

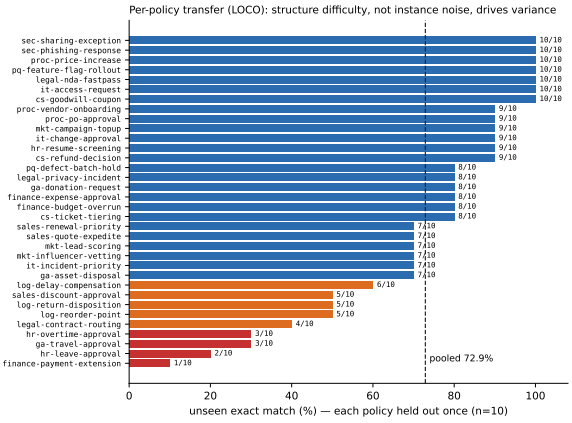

Figure 2: Per-policy transfer under LOCO; every policy is unseen exactly once (n=10 each).

while removing per-request example tokens. Per-policy scores range 1/10–10/10 under LOCO (Figure 2); policies whose conditions embed date/limit arithmetic (leave, overtime, payment extension) sit at the bottom, foreshadowing §5.3.

5.2 What was learned: the counterfactual control

We flip approve↔reject in every rule (and the fallback) of the held-out cases and evaluate **only on records where the flipped policy decides differently from the original**—on this subset, answering from priors or from the original policy scores zero by construction. The tuned model scores **88.0%** (44/50); raw 7B scores 48.0% (Figure 1b). Combined with §5.1, this is direct evidence that fine-tuning taught *application of the in-context policy*, not policy content and not world priors.

5.3 Negative control: the same recipe cannot learn computation

We ran the identical recipe (same trainer, gate, corpus scale: 300 examples) on a *derivation* task from the same product family: regenerate a customer's renewal-pricing files (22 numeric fields) byte-exactly for held-out customers, judged by a deterministic gate.

This is a *control, not a discovery*—routing arithmetic to tools is settled practice. Its role: (i) it closes the "train harder" objection (the CoT-target ablation still fails; the frontier fails the same gate on the same two long-division fields while getting every band and discount rule right; the recorded agent originally passed because it computed with

tools); (ii) field-level scores show what the exact gate hides—inference gets 20–21 of 22 fields, failing precisely where computation, not rule-reading, is required; (iii) it motivates going beyond the standard remedy: a per-run **tool call keeps the LLM in the loop on every execution** (re-deciding which tool to invoke, nondeterministically, with orchestration tokens), whereas compiling the step to the **code tier makes that decision once**, at compile time. The placement rule: derivations → code; policy judgments → trained SLM behind the deterministic gate; declared discretion bands → recommendation + human approver.

5.4 Placement economics: break-even at the second run

Measured on the renewal pipeline in unique tokens (not double-counting the cumulative context an agent resends each turn): the recorded agent session costs 23,614 frontier tokens per run; after compilation and SLM promotion a run consumes **0 frontier tokens** (4,208 tokens on a local model at $0 marginal cost, 7.4× faster, outputs reproduced 7/7 under a deterministic benchmark). Because the compiler's input is a work session the organization had to run once anyway, the frontier-token **break-even is the second run**—compared with the ≈17-transaction break-even reported for optimization-heavy compilation (Trooskens et al., 2026), where the compiler spends LLM tokens searching. Escalations for genuinely new parameter values each cost one frontier call, are cached with upstream-output fingerprints, and amortize to zero on repeats. The hidden denominator is the human cost of declaring the ontology and policy; the pipeline pays off in proportion to a decision's repetition frequency—exactly the regime organizations automate first.

6 Discussion

Why not internalize? A weights-internalized policy answers fast but ages instantly and cannot explain a rule edit. Policy-in-context inverts the maintenance economics: the policy file is the deployment artifact; retraining is only needed when the *skill*, not the policy, drifts.

Beyond policy decisions. Nothing in the recipe is specific to approval workflows: training rows are (document, record, grounded structured answer) triples, and the counterfactual control generalizes to any document class whose semantics can be programmatically perturbed. Runbooks and rubrics are the nearest neighbors; each needs its own deterministic grader, which is the actual bottleneck for widening the claim (Table 3). This paper is also one tier of a larger compiled-pipeline system—recorded agent sessions lowered to code/rule/cache tiers, with the trained-SLM tier occupying the judgment slot behind a promotion gate—whose end-to-end economics (§5.4) motivate but do not depend on the transfer result.

document class	grader needed	status
decision policies	rules engine	**measured (this paper)**
escalation matrices	table lookup	near (catalog cases)
compliance checklists	per-item rules	not started
grading rubrics	answer key	not started
support runbooks	state machine	not started

Table 3: Generalization roadmap for the content-in-context, skill-in-weights recipe: what each document class needs before its transfer can be measured.

Path to prose policies. Our substrate removes entity alignment (rule field names match record keys). The natural next experiment keeps the trained applier fixed and inserts a policy-compilation step (prose → declarative rules) in front—measuring how much of the 72.9% survives noisy compilation, and whether the counterfactual control still holds.

Limitations

Labels and grades come from the same engine; we mitigated with the grounds-hold check, the relaxed metric, the counterfactual control and full pinning, but human-annotated decisions with inter-annotator agreement remain future work. Scale: 34 policies, one 7B base model, one language pair (Korean prompts, mixed-language fields); cluster CIs are honest about this—$[64.7, 80.9]$ is the claim, not 72.9%. Policies are already machine-readable; prose-policy entity alignment is untested. The frontier control uses one frontier model, one prompt format, without tools by construction.

Reproducibility

Corpus generator, trainer (pinned; per-run manifests), evaluation code, all raw results and this draft live in one repository; every table is regen-

erable from a seeded generator and stored model outputs.

References

Peter Belcak et al. 2025. Small language models are the future of agentic ai. ArXiv:2506.02153.

Tianzhe Chu et al. 2025. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. ArXiv:2501.17161.

Lingxiao Diao, Xinyue Xu, Wanxuan Sun, Cheng Yang, and Zhuosheng Zhang. 2025. Guidebench: Benchmarking domain-oriented guideline following for llm agents. ArXiv:2505.11368.

Nouha Dziri et al. 2023. Faith and fate: Limits of transformers on compositionality. ArXiv:2305.18654, NeurIPS 2023.

Luyu Gao et al. 2022. Pal: Program-aided language models. ArXiv:2211.10435.

Kung-Hsiang Huang, Akshara Prabhakar, Onkar Thorat, Divyansh Agarwal, Prafulla Kumar Choubey, Yixin Mao, Silvio Savarese, Caiming Xiong, and Chien-Sheng Wu. 2025. Crmarena-pro: Holistic assessment of llm agents across diverse business scenarios and interactions. ArXiv:2505.18878.

Joseph Marvin Imperial and Harish Tayyar Madabushi. 2025. Scaling policy compliance assessment in language models with policy reasoning traces. ArXiv:2509.23291.

Object Management Group. 2024. Decision model and notation (dmn) v1.5. OMG standard.

Geert Trooskens, Aaron Karlsberg, Anmol Sharma, Lamara De Brouwer, Max Van Puyvelde, Matthew Young, John Thickstun, Gil Alterovitz, and Walter A. De Brouwer. 2026. Compiled ai: Deterministic code generation for llm-based workflow automation. ArXiv:2604.05150.

Zhenhailong Wang, Jiateng Liu, Amin Fazel, Ritesh Sarkhel, Xing Fan, Xiang Li, Chenlei Guo, Heng Ji, and Ruhi Sarikaya. 2025. Multimodal policy internalization for conversational agents. ArXiv:2510.09474.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. ArXiv:2406.12045.

Tao Yu, Hao Wang, Changyu Li, et al. 2026. Beyond the all-in-one agent: Benchmarking role-specialized multi-agent collaboration in enterprise workflows. ArXiv:2605.08761.

Ruiwen Zhou, Wenyue Hua, Liangming Pan, Sitao Cheng, Xiaobao Wu, En Yu, and William Yang Wang. 2024. Rulearena: A benchmark for rule-guided reasoning with llms in real-world scenarios. ArXiv:2412.08972.